

Project: Deploying a Node.js Web Application on AWS Free Tier Using Docker & Kubernetes with CI/CD Pipeline

Submission Type: Individual

Due Date: June 1, 2026 (Before 12 AM)

Project Overview

You are tasked with building a simple Node.js web application, containerizing it with Docker, storing the image in Amazon ECR, and deploying it to a Kubernetes cluster running on a single EC2 t2.micro instance using Minikube. The application must be publicly accessible via the EC2 instance's public IP address – all within the AWS Free Tier limits. No payment is required at any stage.

The application must display basic dynamic information (timestamp, container ID and visitor counter) and be publicly accessible via a web browser.

Additional Task: You must create a **CI/CD pipeline using GitHub Actions** that automatically builds the Docker image, pushes it to Amazon ECR, and deploys (or redeploys) the application to your Minikube cluster whenever code is pushed to the main branch of your GitHub repository.

Primary Goal: Understand the end-to-end workflow of containerization, container registry, Kubernetes deployment on a single-node cluster, and CI/CD automation – entirely within free tier constraints.

Tools: AWS Free Tier, Docker, Kubernetes (Minikube on EC2), GitHub, GitHub Actions (CI/CD)

Deliverables (What to Submit)

#	Deliverable	Format	Submission Method
1	Public Application URL (EC2 IP + NodePort)	Live link	Paste in report
2	Source Code Repository (Node.js app + Dockerfile + Kubernetes YAML + GitHub Actions workflow)	GitHub (Public)	Link in report

#	Deliverable	Format	Submission Method
3	Project Report (Max 8 pages)	PDF	Moellim Upload
4	Demonstration Video (5 minutes max) (details given in the end)	YouTube (Unlisted) or Google Drive	Link in report
5	Screenshots (Running app, ECR image, EC2 instance, Minikube cluster, deployed pods, GitHub Actions workflow success)	Embedded in report	Paste in report

Project Report Structure (8 Pages Max)

Your report must contain the following sections:

1. **Title Page** – Project title, your name, roll number, date
2. **Introduction** – What problem does this solve? Why containers + Kubernetes? Why CI/CD?
3. **Architecture Diagram** – Updated diagram showing GitHub → GitHub Actions → ECR → EC2 + Minikube
4. **Step-by-Step Implementation** – Describe each step you followed (use the guideline below)
5. **CI/CD Pipeline Design** – Explanation of GitHub Actions workflow (triggers, jobs, secrets, deployment strategy)
6. **Challenges Faced** – Minimum 3 issues and how you resolved them (include at least one CI/CD related challenge)
7. **Cost Analysis** – Table showing why you paid \$0 (EC2 free tier, ECR free storage, Minikube is free, GitHub Actions free tier)
8. **Screenshots** – Minimum 8 screenshots with captions (including GitHub Actions workflow runs)

9. **Conclusion & Learnings** – What you learned about cloud computing, containers, Kubernetes, and CI/CD automation
-

Step-by-Step Implementation Guideline (Updated with CI/CD)

Prerequisites (Install on Your Laptop)

Before starting, install these free tools:

- Node.js (version 18 or higher) – nodejs.org
- Docker Desktop – docker.com/products/docker-desktop
- AWS CLI – aws.amazon.com/cli
- Git – git-scm.com
- AWS Account (Free Tier) – aws.amazon.com/free
- GitHub Account – github.com

Phase 1: Create a Simple Node.js Web Application

Phase 2: Containerize the Application with Docker

Phase 3: Push Image to Amazon ECR

Phase 4: Launch EC2 Instance (Free Tier)

Phase 5: Connect to EC2 and Install Docker & Minikube

Phase 6: Authenticate EC2 to ECR and Pull Image

Phase 7: Deploy to Kubernetes (Minikube)

Phase 8: Create GitHub Actions CI/CD Pipeline (NEW)

- Workflow steps:
 1. Checkout code
 2. Set up Docker Buildx
 3. Configure AWS credentials
 4. Log in to Amazon ECR
 5. Build Docker image
 6. Tag and push image to ECR
 7. SSH into EC2 instance
 8. Pull new image on EC2
 9. Apply or restart Kubernetes deployment (`kubect1 rollout restart`)

Phase 9: Testing & Verification

Phase 10: Clean Up Resources

Demonstration Video Requirements (5 minutes max)

Timestamp	Content
0:00 – 0:30	Introduction – your name, project title, explanation of free tier constraints and CI/CD goal
0:30 – 1:30	Show code on GitHub (Node.js app + Dockerfile + Kubernetes YAML + GitHub Actions workflow YAML)
1:30 – 2:00	Show Docker image in ECR (AWS Console) and GitHub Actions successful workflow run
2:00 – 2:30	Show EC2 instance running (AWS Console) and SSH connection
2:30 – 3:30	LIVE DEMO – Run <code>kubect1 get pods</code> and <code>kubect1 get services</code> , open browser, show running app via EC2 IP and NodePort
3:30 – 4:00	DEMO CI/CD – Make a small code change (e.g., change webpage text), push to GitHub, show workflow automatically triggering, refresh browser to see update deployed

Timestamp	Content
4:00 – 4:30	Show scaling demo (<code>kubectl scale --replicas=2</code>) and show <code>/health</code> endpoint
4:30 – 5:00	Conclusion – what you learned about CI/CD automation, and demonstrate termination of resources (cost safety)

Additional CI/CD Requirements (To Be Included in Report)

Your report must be explicitly addressed:

1. **GitHub Actions Workflow File** – Provide the complete YAML code in the appendix
2. **Secrets Management** – List which secrets were stored in GitHub and why
3. **Trigger Strategy** – Explain when the pipeline runs (e.g., `push to main`, `pull request`)
4. **Deployment Strategy** – How does the pipeline update the running application? (e.g., `kubectl rollout restart` or `kubectl apply`)
5. **Failure Handling** – What happens if the build or deployment fails?

-Good Luck-